Part 2: 2
Bài 1: Chạy một container đơn giản với Docker Compose 2
Bài 2: Chạy MySQL với Docker Compose 3
Bài 3: Kết nối MySQL với PHPMyAdmin 4
Bài 4: Chạy ứng dụng Node.js với Docker Compose 5
Bài 5: Chạy Redis với Docker Compose 6
Bài 6: Chạy WordPress với MySQL 7
Bài 7: Chạy MongoDB với Docker Compose 8
Bài 8: Kết nối nhiều dịch vụ với Docker Compose 9
Bài 9: Chạy ứng dụng Python Flask với Docker Compose 10
Bài 10: Lưu trữ dữ liệu với Docker Volumes 11
Bài 11: Chạy dịch vụ Postgres với Adminer 12
Bài 12: Giám sát container với Prometheus và Grafana 13
Bài 13: Chạy ứng dụng React với Nginx 14
Bài 14: Cấu hình mạng riêng giữa các container 15
Bài 15: Giới hạn tài nguyên cho container 16
Part 3: 17
Bài tập 1: Triển khai WordPress với MySQL 17
Bài tập 2: Ứng dụng Node.js + MongoDB 18
Bài tập 4: Prometheus + Grafana Monitoring 19
Bài tập 5: Multi-tier Voting App 20
Bài tập 7: Elasticsearch + Kibana 21
Bài tập 8: Django + Celery + Redis 22
Bài tập 10: Traefik as Reverse Proxy 24

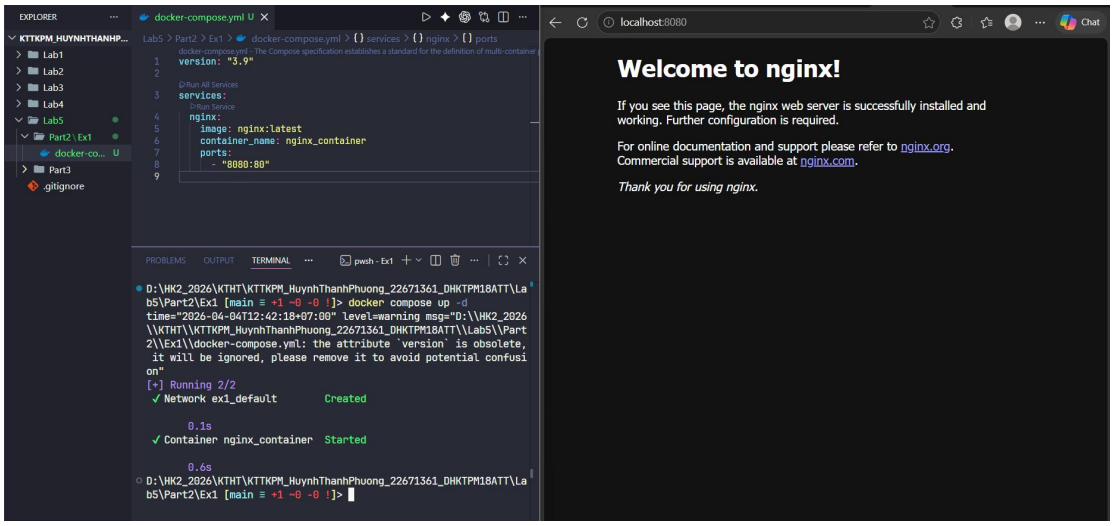
Part 2:

Bài 1: Chạy một container đơn giản với Docker Compose

Yêu cầu:

Tạo một container chạy Nginx bằng Docker Compose.

Map cổng 8080 của máy host với cổng 80 của container.

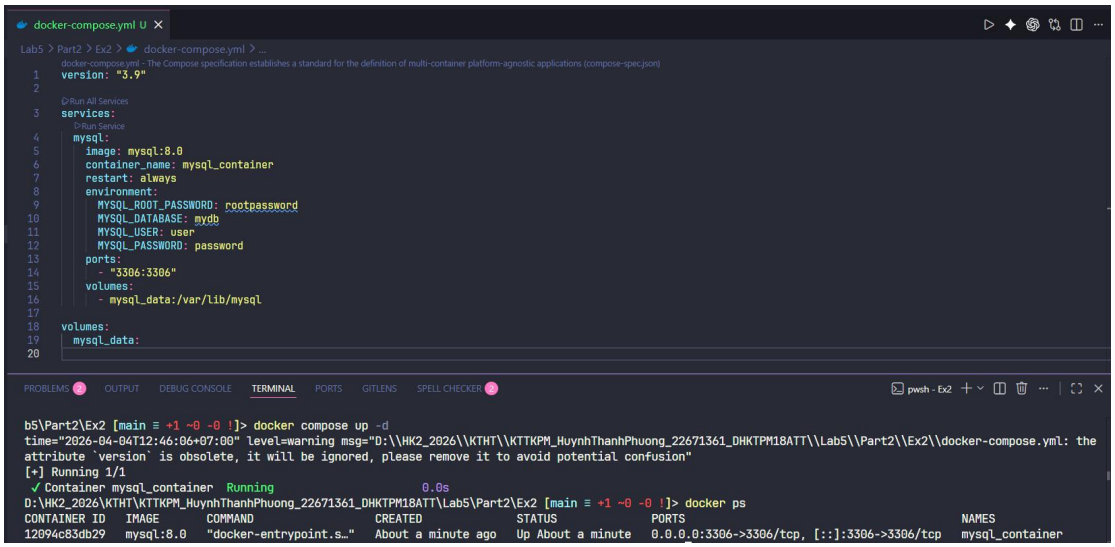


Bài 2: Chạy MySQL với Docker Compose

Yêu cầu:

Tạo một container chạy MySQL phiên bản 8.0.

Đặt username là user, password là password và database là mydb

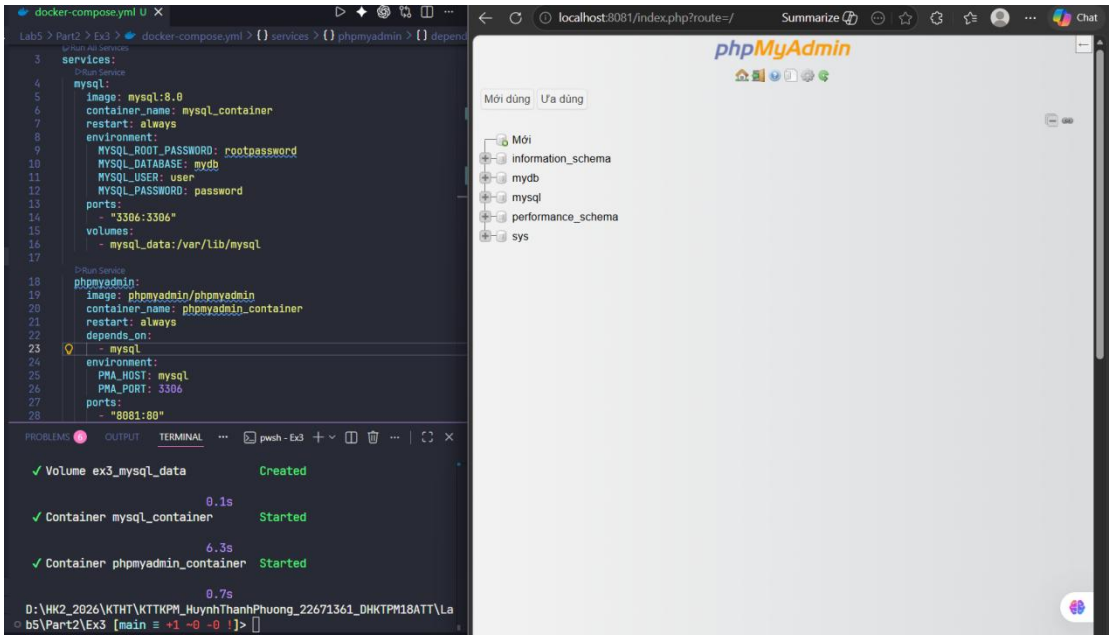


Bài 3: Kết nối MySQL với PHPMyAdmin

Yêu cầu:

Chạy MySQL và PHPMyAdmin với Docker Compose.

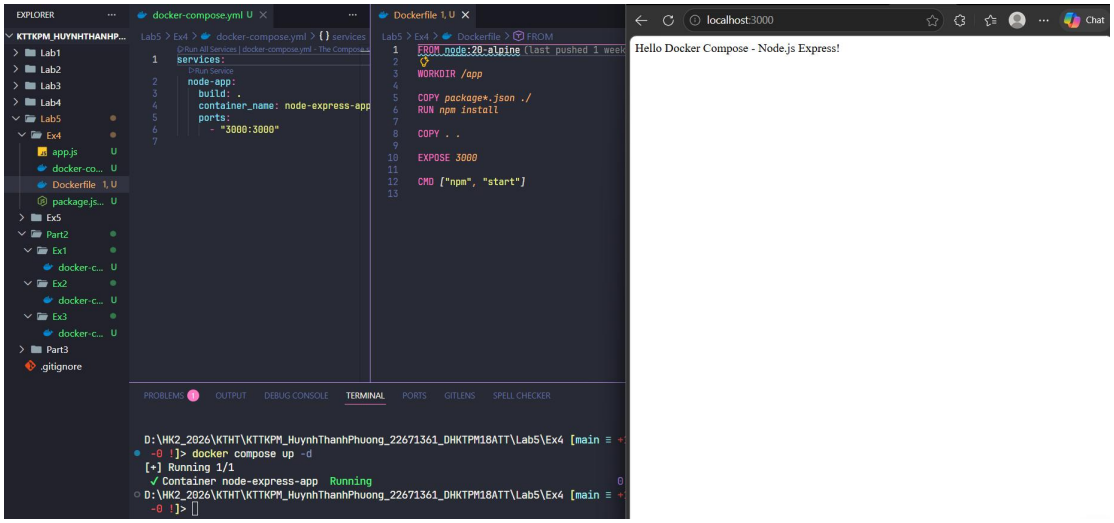
PHPMyAdmin chạy trên cổng 8081.



Bài 4: Chạy ứng dụng Node.js với Docker Compose

Yêu cầu:

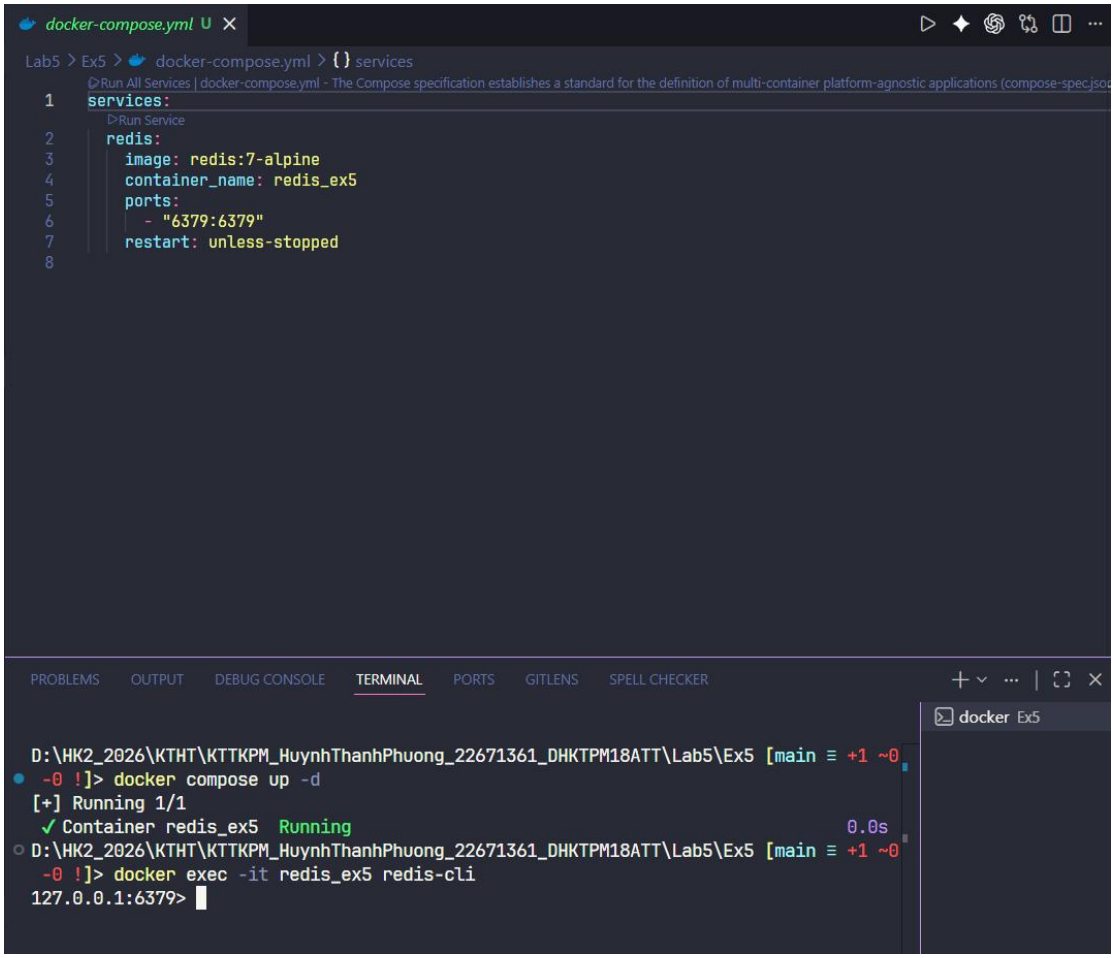
Chạy một ứng dụng Node.js đơn giản với Express.



Bài 5: Chạy Redis với Docker Compose

Yêu cầu:

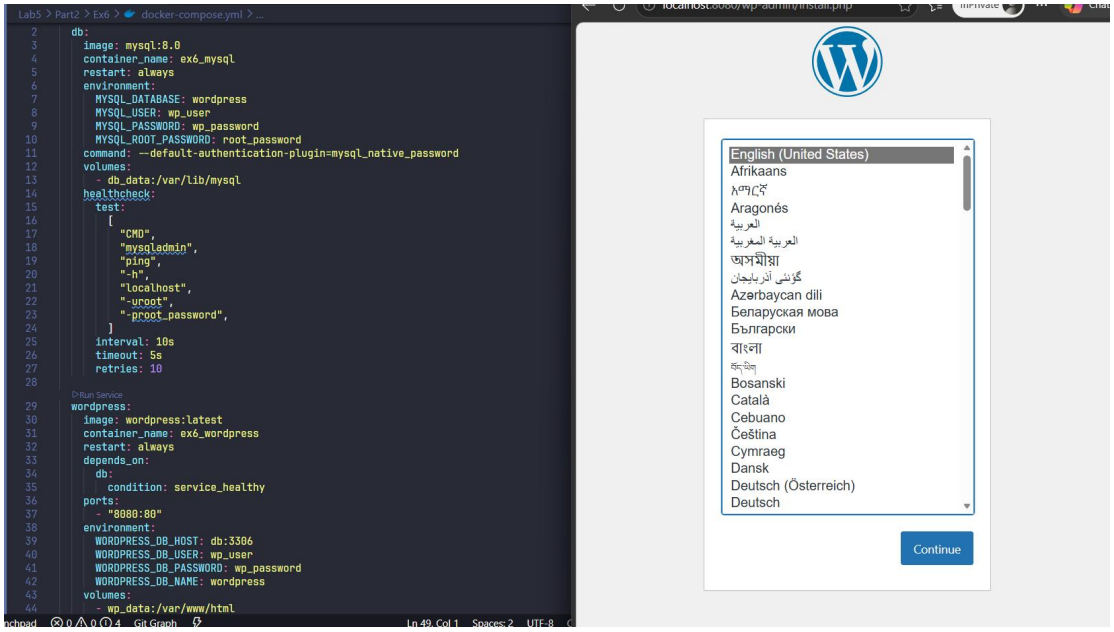
Chạy một container Redis trên cổng 6379.



Bài 6: Chạy WordPress với MySQL

Yêu cầu:

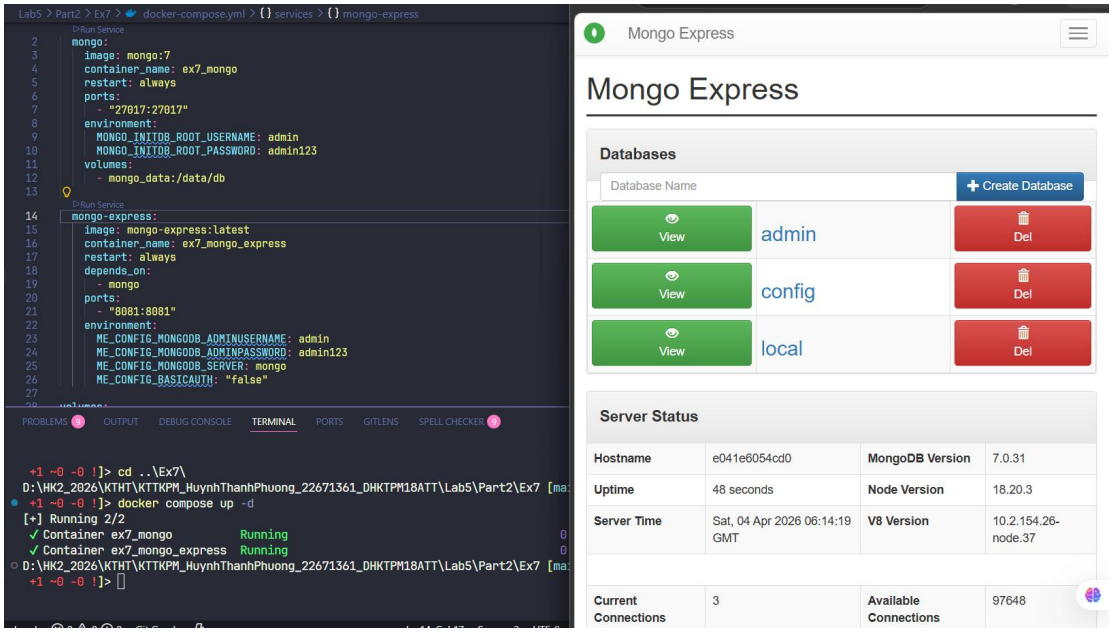
Chạy WordPress với MySQL bằng Docker Compose.



Bài 7: Chạy MongoDB với Docker Compose

Yêu cầu:

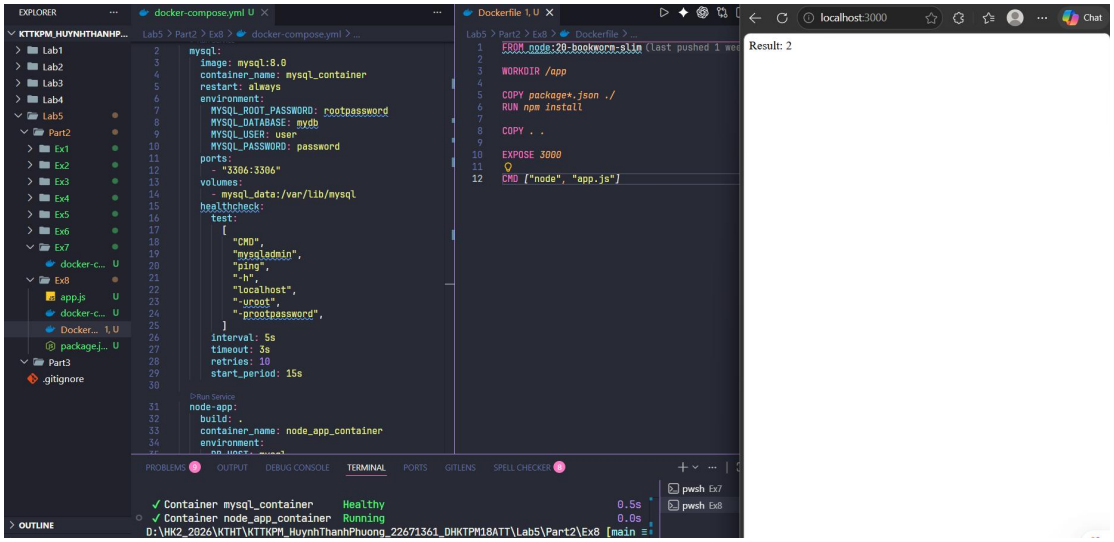
Chạy MongoDB và Mongo Express để quản lý.



Bài 8: Kết nối nhiều dịch vụ với Docker Compose

Yêu cầu:

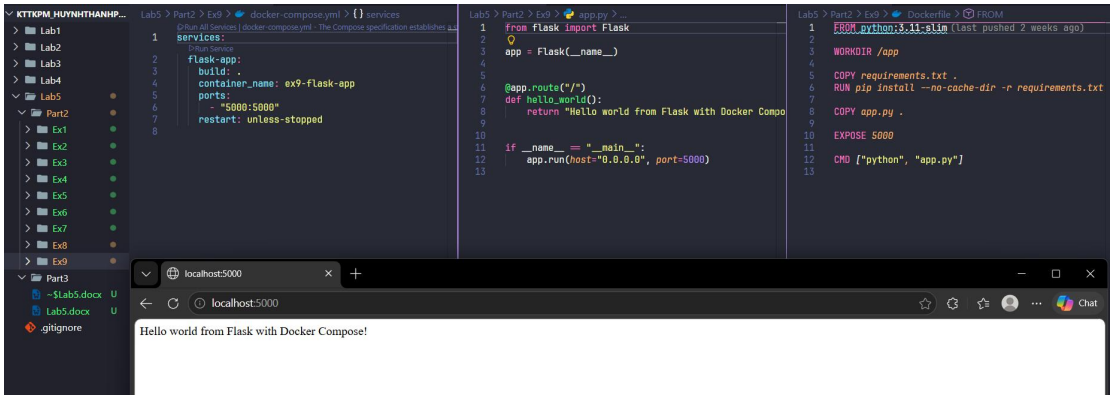
Chạy Node.js kết nối với MySQL.



Bài 9: Chạy ứng dụng Python Flask với Docker Compose

Yêu cầu:

Chạy ứng dụng Flask đơn giản với Docker Compose.



Bài 10: Lưu trữ dữ liệu với Docker Volumes

Yêu cầu:

Chạy MySQL và gắn volume để dữ liệu không bị mất.

docker-compose.yml U

Lab5 > Part2 > Ex10 > docker-compose.yml > { } services > { } mysql > { } environment > MYSQL_PASSWORD

Run All Services | docker-compose.yml - The Compose specification establishes a standard for the definition of multi-container platform-agnostic ap

```yaml
services:
  mysql:
    image: mysql:8.0
    container_name: ex10-mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: demo
      MYSQL_USER: appuser
      MYSQL_PASSWORD: app123
    ports:
      - "3307:3306"
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 1

pwsh

```
D:\HK2_2026\KTHT\KTTKPM_HuynhThanhPhuong_22671361_DHKTPM18ATT [main
≡ +1 ~0 -0 !]> docker volume inspect ex10_mysql_data
[
    {
        "CreatedAt": "2026-04-04T06:40:06Z",
        "Driver": "local",
        "Labels": {
            "com.docker.compose.config-hash": "ea9c218692950a18af0d6
72b860ef5b7d7d23aff64f6a0337b99a55f5cbd881e",
            "com.docker.compose.project": "ex10",
            "com.docker.compose.version": "2.40.3",
            "com.docker.compose.volume": "mysql_data"
        },
        "Mountpoint": "/var/lib/docker/volumes/ex10_mysql_data/_data
",
        "Name": "ex10_mysql_data",
```

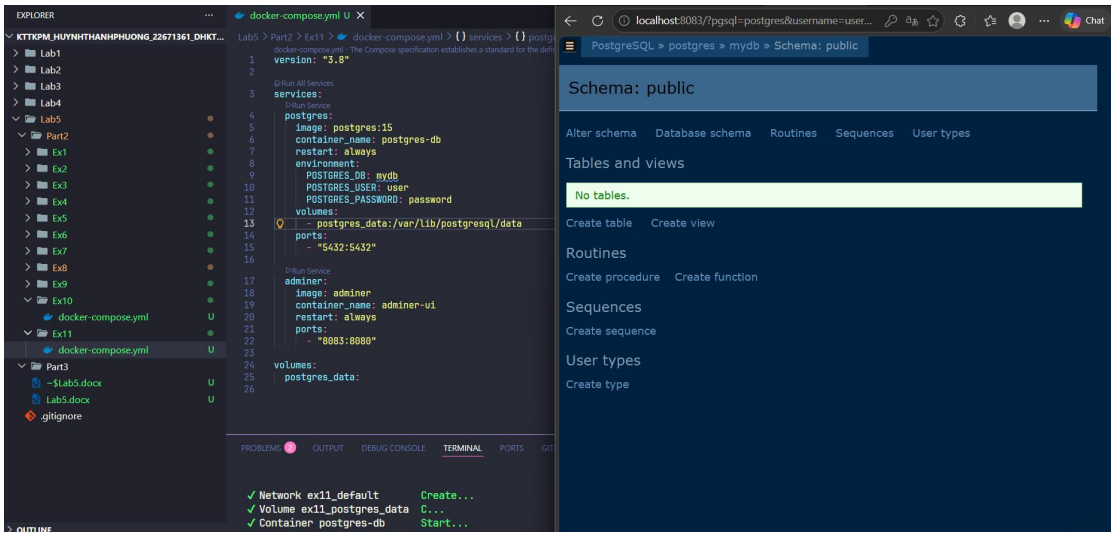
Bài 11: Chạy dịch vụ Postgres với Adminer

Yêu cầu:

Chạy PostgreSQL và Adminer (công cụ quản lý database) bằng Docker Compose.

PostgreSQL phải có database tên mydb, user là user, password là password.

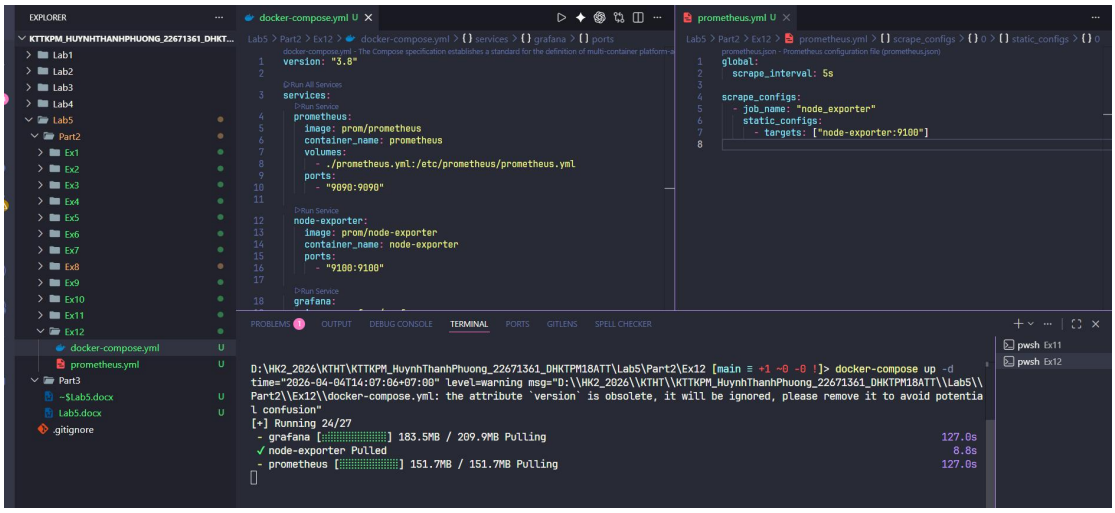
Adminer chạy trên cổng 8083

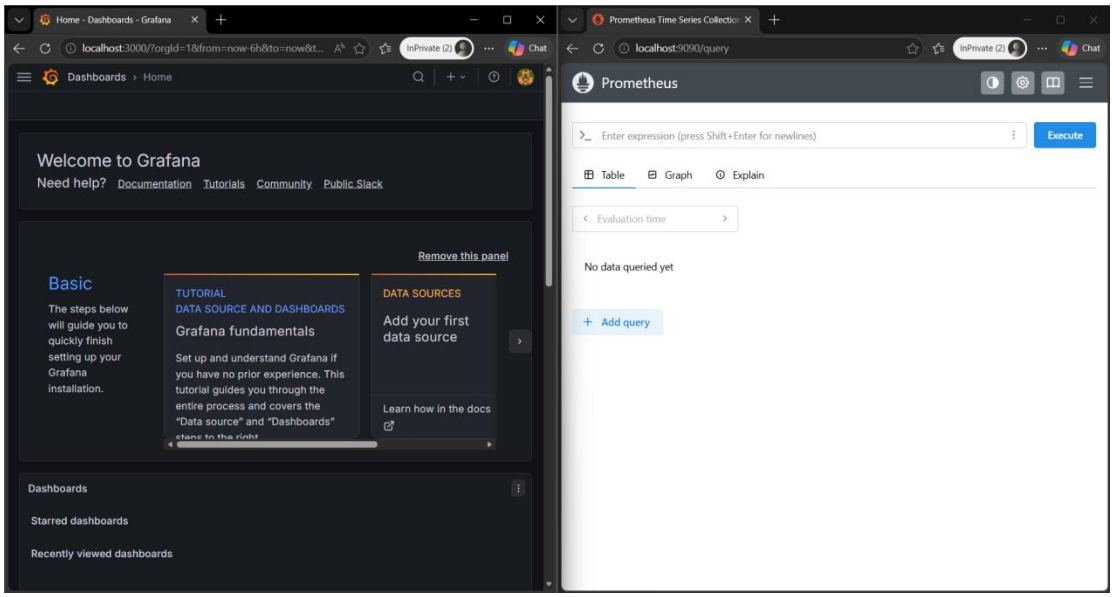


Bài 12: Giám sát container với Prometheus và Grafana

Yêu cầu:

Chạy Prometheus, Grafana và Node Exporter bằng Docker Compose để giám sát hệ thống.

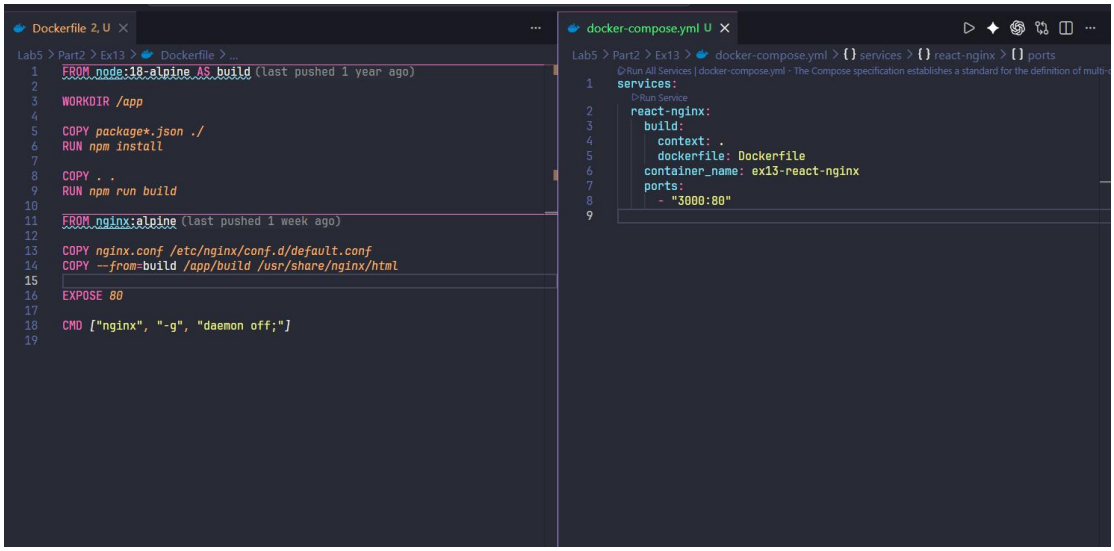


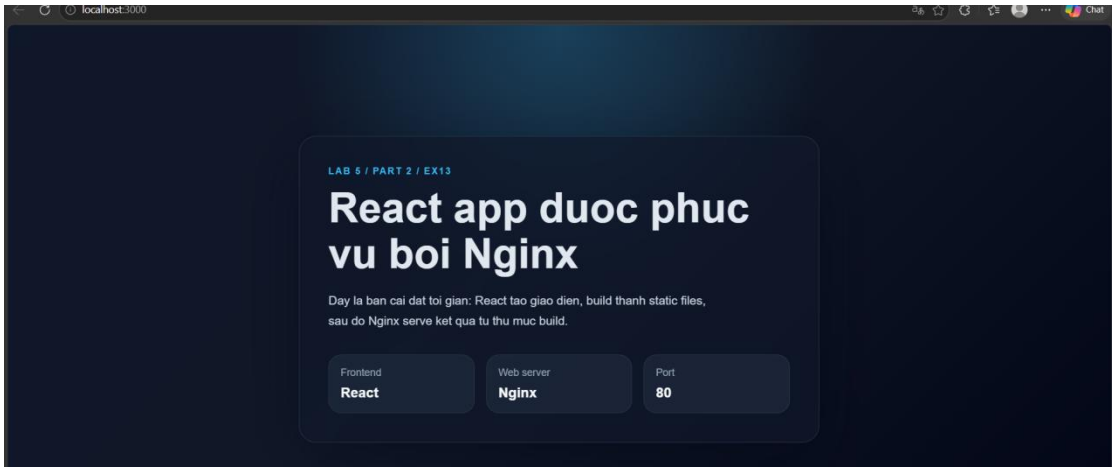


Bài 13: Chạy ứng dụng React với Nginx

Yêu cầu:

Chạy một ứng dụng React và serve nó bằng Nginx.

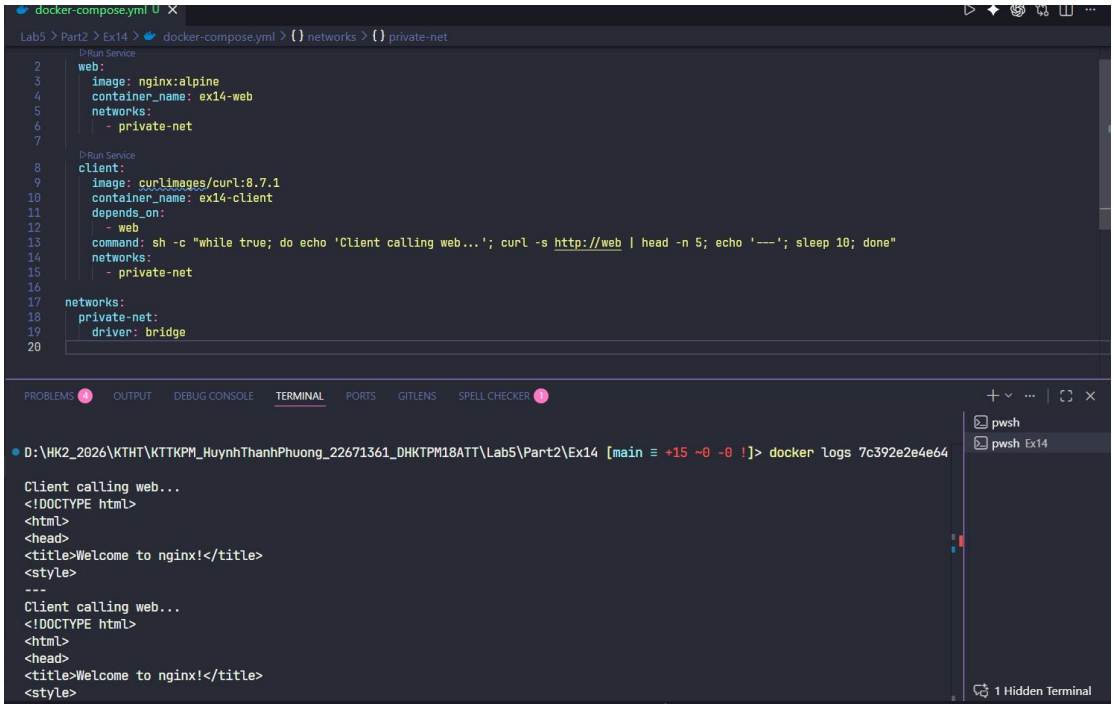




Bài 14: Cấu hình mạng riêng giữa các container

Yêu cầu:

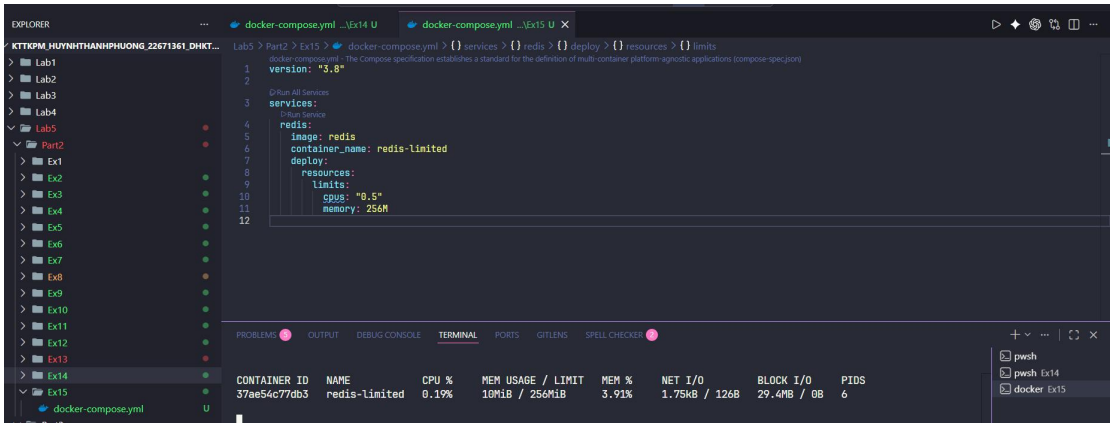
Chạy 2 container có thể giao tiếp với nhau trong một mạng riêng.



Bài 15: Giới hạn tài nguyên cho container

Yêu cầu:

Giới hạn CPU và RAM cho một container Redis.



Part 3:

Bài tập 1: Triển khai WordPress với MySQL

Mục tiêu: Tạo stack WordPress kết nối với MySQL, sử dụng volumes để lưu trữ dữ liệu.

Yêu cầu:

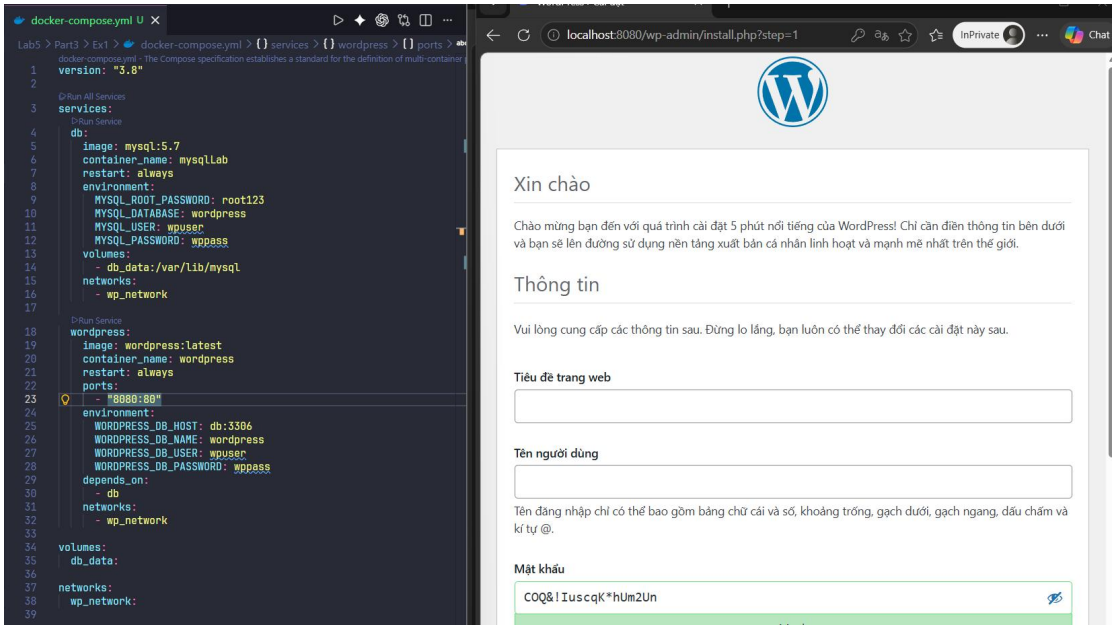
1. Sử dụng image wordpress:latest (port 80).
2. Sử dụng image mysql:5.7 (port 3306).
3. Volume cho database (/var/lib/mysql).
4. Biến môi trường cho MySQL:

MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD

Gợi ý:

WordPress cần khai báo depends_on MySQL.

Sử dụng network tùy chỉnh để kết nối giữa 2 service.

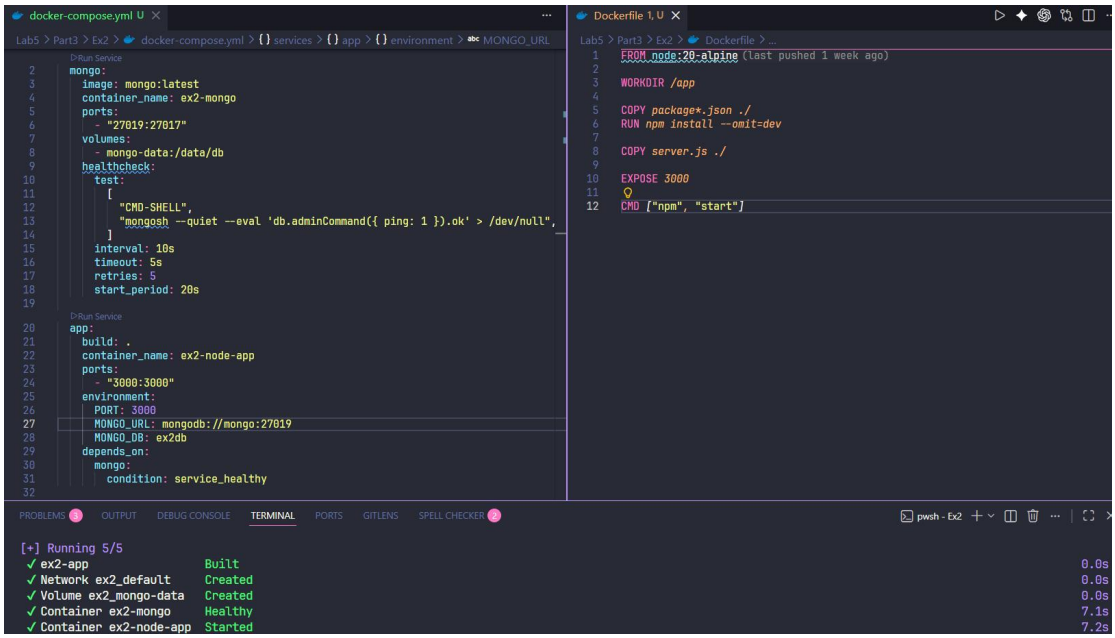


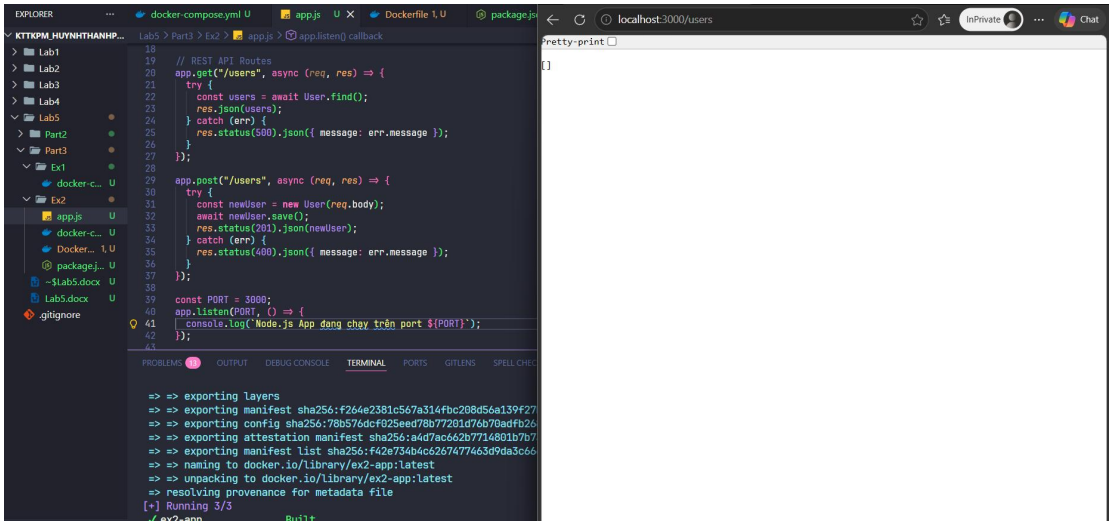
Bài tập 2: Ứng dụng Node.js + MongoDB

Mục tiêu: Triển khai ứng dụng Node.js (lưu dữ liệu vào MongoDB) và MongoDB với volume.

Yêu cầu:

1. Viết Dockerfile cho ứng dụng Node.js (ví dụ: REST API đơn giản).
2. Sử dụng image mongo:latest (port 27017).
3. Volume cho MongoDB (/data/db).
4. Đảm bảo Node.js service khởi động sau MongoDB (depends_on + healthcheck).



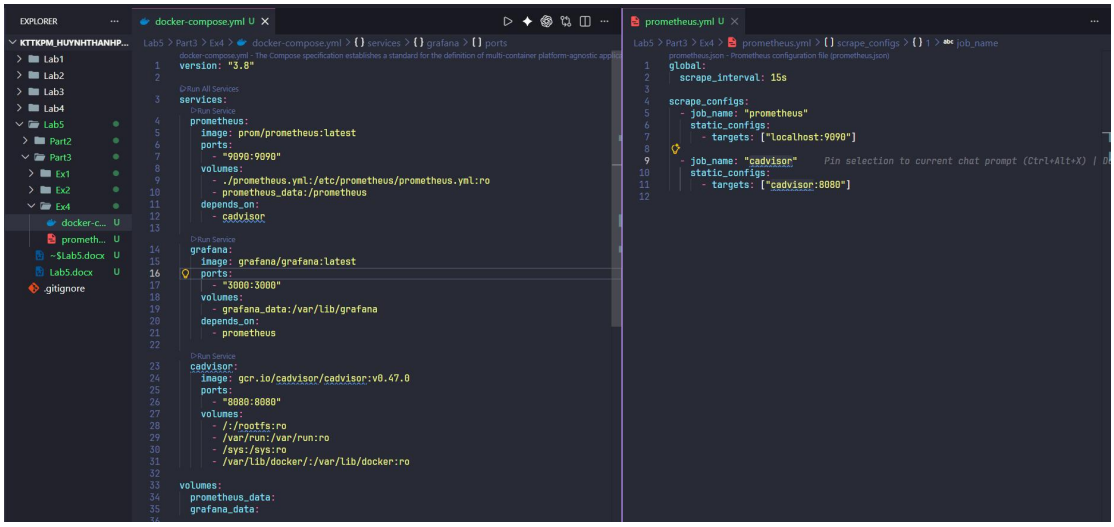


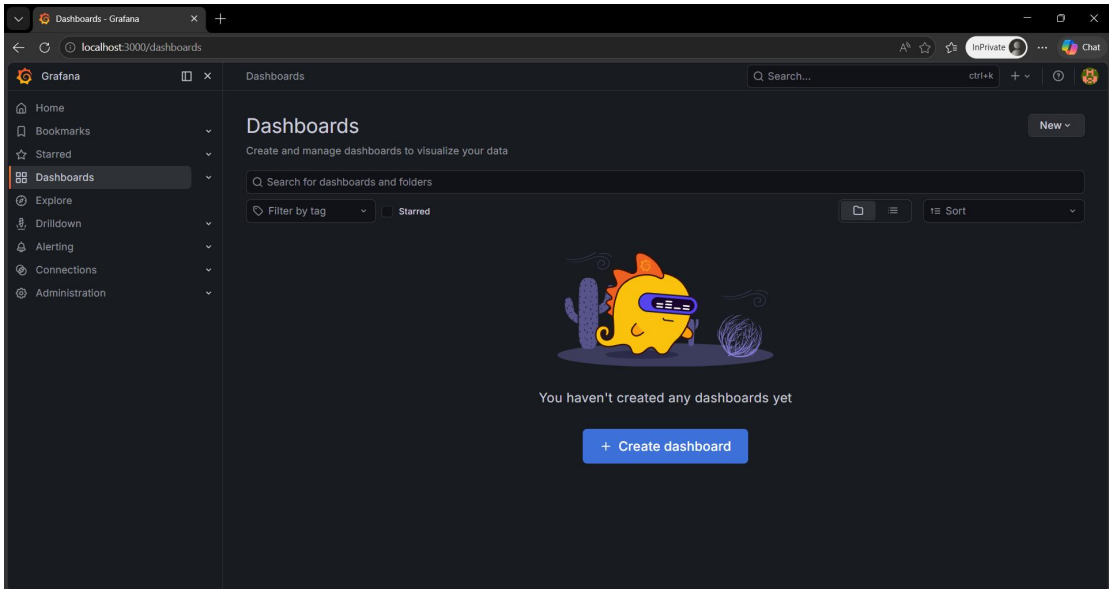
Bài tập 4: Prometheus + Grafana Monitoring

Mục tiêu: Giám sát Docker containers dùng Prometheus và Grafana.

Yêu cầu:

1. Service Prometheus (port 9090) với file cấu hình thu thập metrics từ Docker.
2. Service Grafana (port 3000) kết nối đến Prometheus.
3. Volume để lưu dữ liệu Prometheus và Grafana.



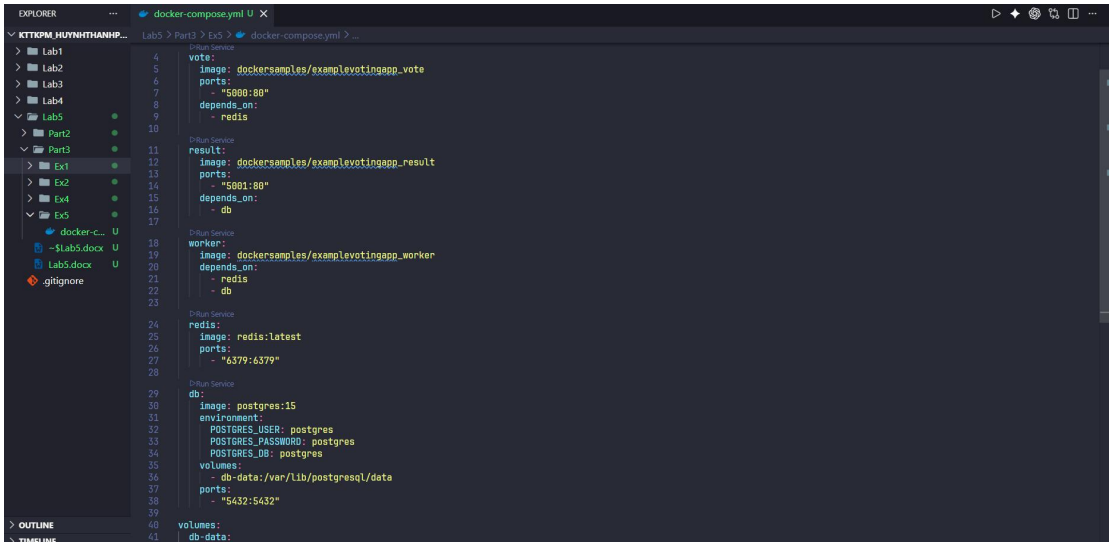


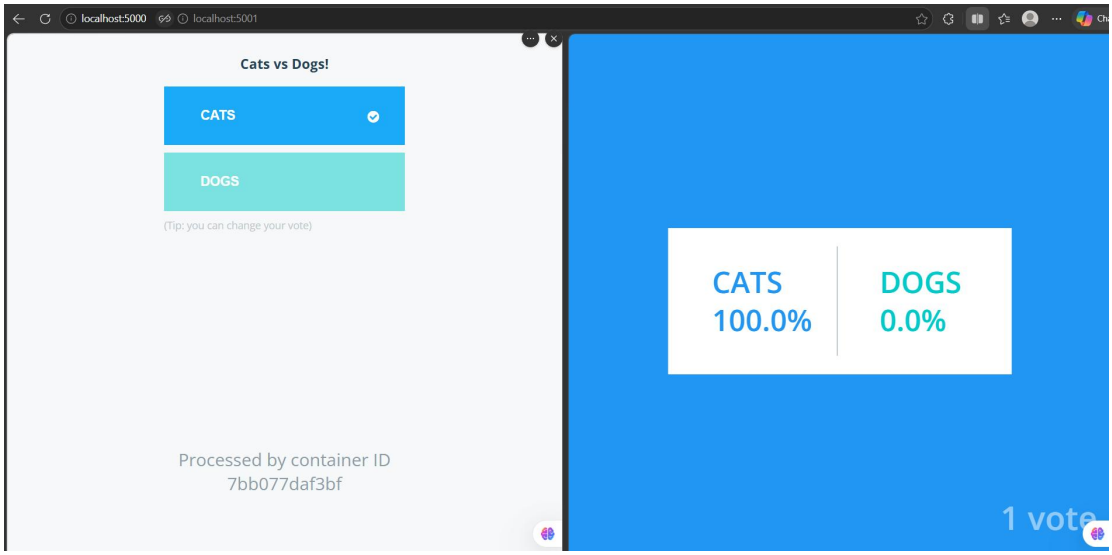
Bài tập 5: Multi-tier Voting App

Mục tiêu: Triển khai ứng dụng voting gồm 5 services (Tham khảo từ Docker Docs).

Yêu cầu:

1. Frontend: vote (Python, port 5000).
2. Backend: result (Node.js, port 5001).
3. Redis (lưu tạm vote).
4. Worker (Java) xử lý vote từ Redis sang DB.
5. Postgres (lưu kết quả).



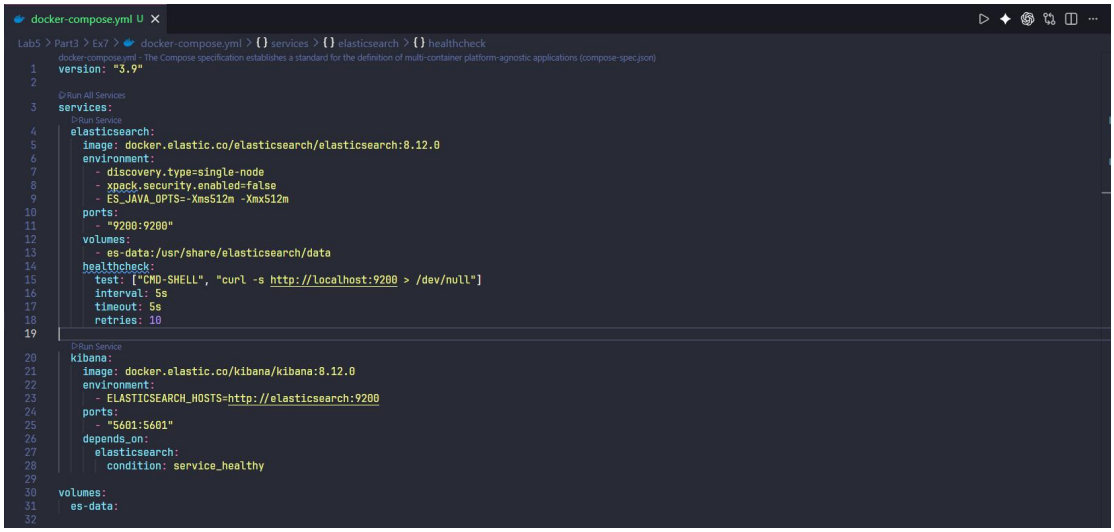


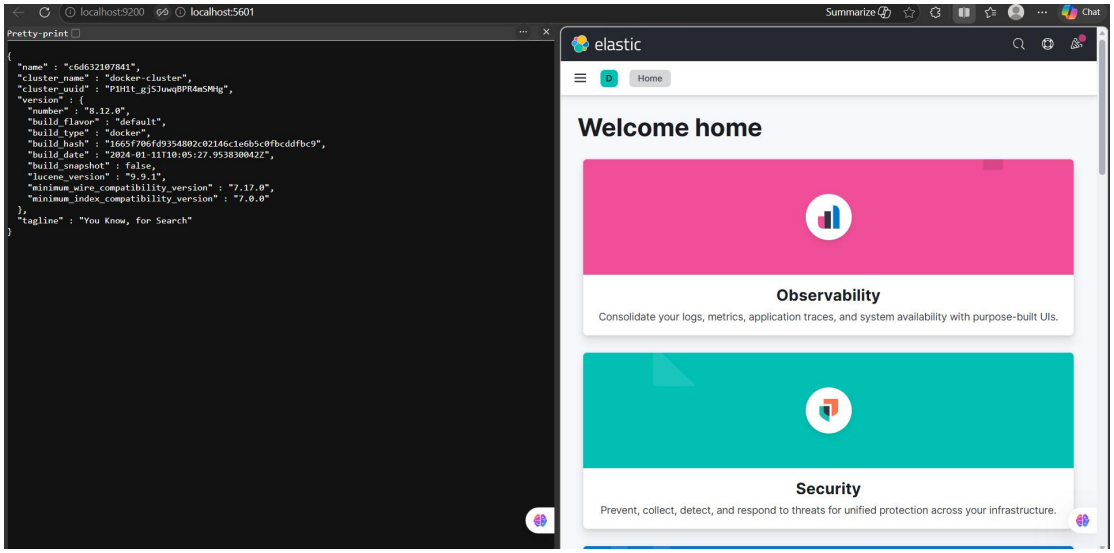
Bài tập 7: Elasticsearch + Kibana

Mục tiêu: Triển khai ELK stack đơn giản.

Yêu cầu:

1. Service Elasticsearch (port 9200) với volume.
2. Service Kibana (port 5601) kết nối với Elasticsearch.
3. Thiết lập environment variables cho credentials.





Bài tập 8: Django + Celery + Redis

Mục tiêu: Triển khai Django với Celery worker và Redis làm message broker.

Yêu cầu:

1. Django app (port 8000).
2. Celery worker chạy song song.
3. Redis service cho task queue.

Dockerfile 1, U

Lab5 > Part3 > Ex8 > config > Dockerfile > FROM

```
FROM python:3.11-slim (last pushed 16 hours ago)

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

docker-compose.yml U

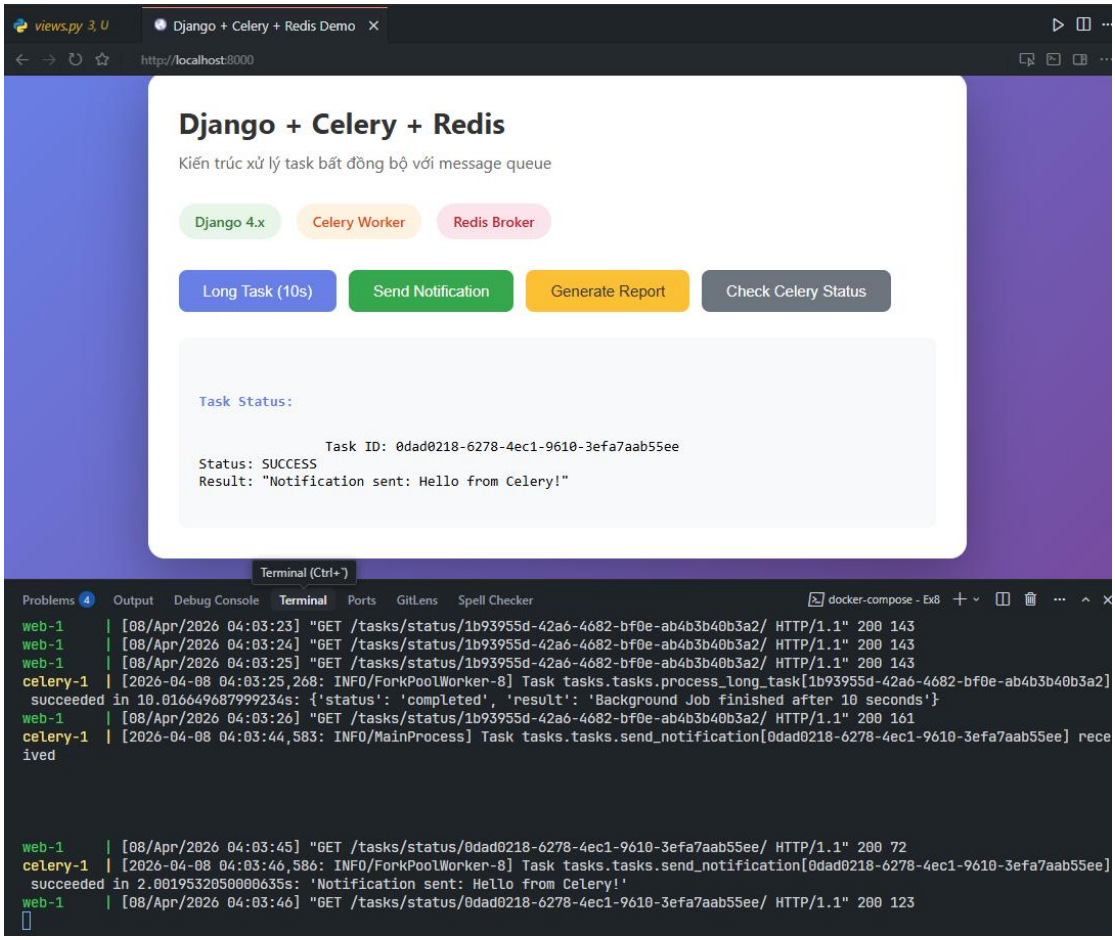
Lab5 > Part3 > Ex8 > docker-compose.yml > ...

```
version: "3.8"

Run All Services
services:
  Run Service
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  Run Service
  web:
    build:
      context: ./config
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    environment:
      - CELERY_BROKER_URL=redis://redis:6379/0
      - CELERY_RESULT_BACKEND=redis://redis:6379/0
      - DEBUG=True
    depends_on:
      redis:
        condition: service_healthy
    command: python manage.py runserver 0.0.0.0:8000

  Run Service
  celery:
    build:
      context: ./config
      dockerfile: Dockerfile
    environment:
      - CELERY_BROKER_URL=redis://redis:6379/0
      - CELERY_RESULT_BACKEND=redis://redis:6379/0
    depends_on:
      redis:
        condition: service_healthy
    command: celery -A config worker --loglevel=info
```


views.py 3, U
Django + Celery + Redis Demo
http://localhost:8000
Django + Celery + Redis
Kiến trúc xử lý task bất đồng bộ với message queue
Django 4.x
Celery Worker
Redis Broker
Long Task (10s)
Send Notification
Generate Report
Check Celery Status
Task Status:
Task ID: 0dad0218-6278-4ec1-9610-3efa7aab55ee
Status: SUCCESS
Result: "Notification sent: Hello from Celery!"
Terminal (Ctrl+`)
Problems 4
Output
Debug Console
Terminal
Ports
GitLens
Spell Checker
docker-compose - Ex8
web-1 | [08/Apr/2026 04:03:23] "GET /tasks/status/1b93955d-42a6-4682-bf0e-ab4b3b40b3a2/ HTTP/1.1" 200 143
web-1 | [08/Apr/2026 04:03:24] "GET /tasks/status/1b93955d-42a6-4682-bf0e-ab4b3b40b3a2/ HTTP/1.1" 200 143
web-1 | [08/Apr/2026 04:03:25] "GET /tasks/status/1b93955d-42a6-4682-bf0e-ab4b3b40b3a2/ HTTP/1.1" 200 143
celery-1 | [2026-04-08 04:03:25,268: INFO/ForkPoolWorker-8] Task tasks.tasks.process_long_task[1b93955d-42a6-4682-bf0e-ab4b3b40b3a2]
succeeded in 10.016649687999234s: {'status': 'completed', 'result': 'Background Job finished after 10 seconds'}
web-1 | [08/Apr/2026 04:03:26] "GET /tasks/status/1b93955d-42a6-4682-bf0e-ab4b3b40b3a2/ HTTP/1.1" 200 161
celery-1 | [2026-04-08 04:03:44,583: INFO/MainProcess] Task tasks.tasks.send_notification[0dad0218-6278-4ec1-9610-3efa7aab55ee] rece
ived
web-1 | [08/Apr/2026 04:03:45] "GET /tasks/status/0dad0218-6278-4ec1-9610-3efa7aab55ee/ HTTP/1.1" 200 72
celery-1 | [2026-04-08 04:03:46,586: INFO/ForkPoolWorker-8] Task tasks.tasks.send_notification[0dad0218-6278-4ec1-9610-3efa7aab55ee]
succeeded in 2.0019532050000635s: 'Notification sent: Hello from Celery!'
web-1 | [08/Apr/2026 04:03:46] "GET /tasks/status/0dad0218-6278-4ec1-9610-3efa7aab55ee/ HTTP/1.1" 200 123

Bài tập 10: Traefik as Reverse Proxy

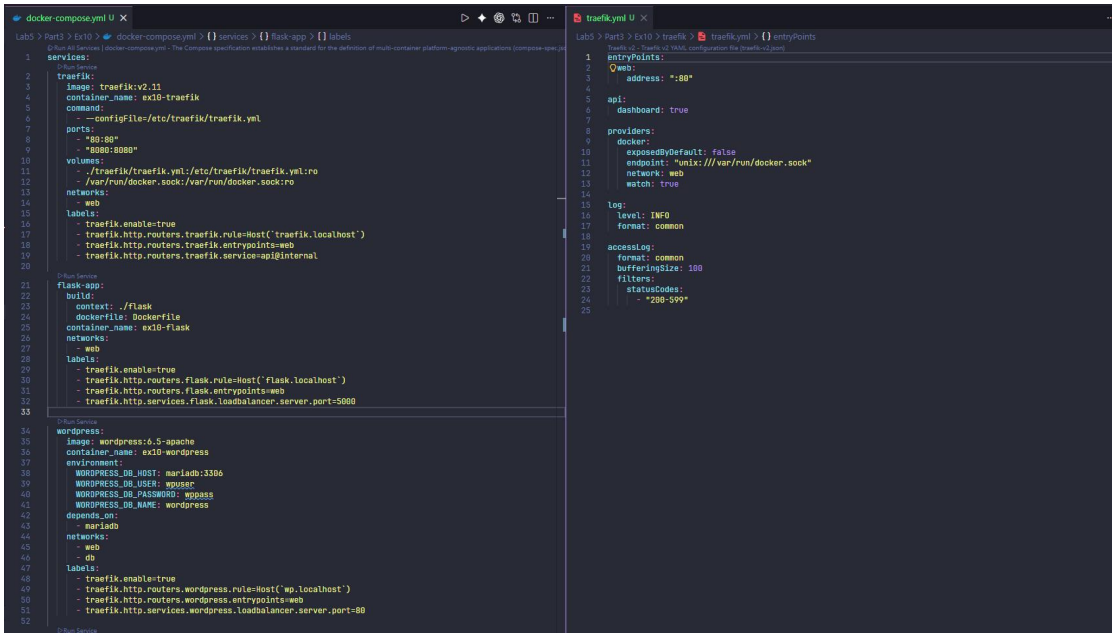
Mục tiêu: Dùng Traefik để định tuyến request đến các service (Flask, WordPress, etc.).

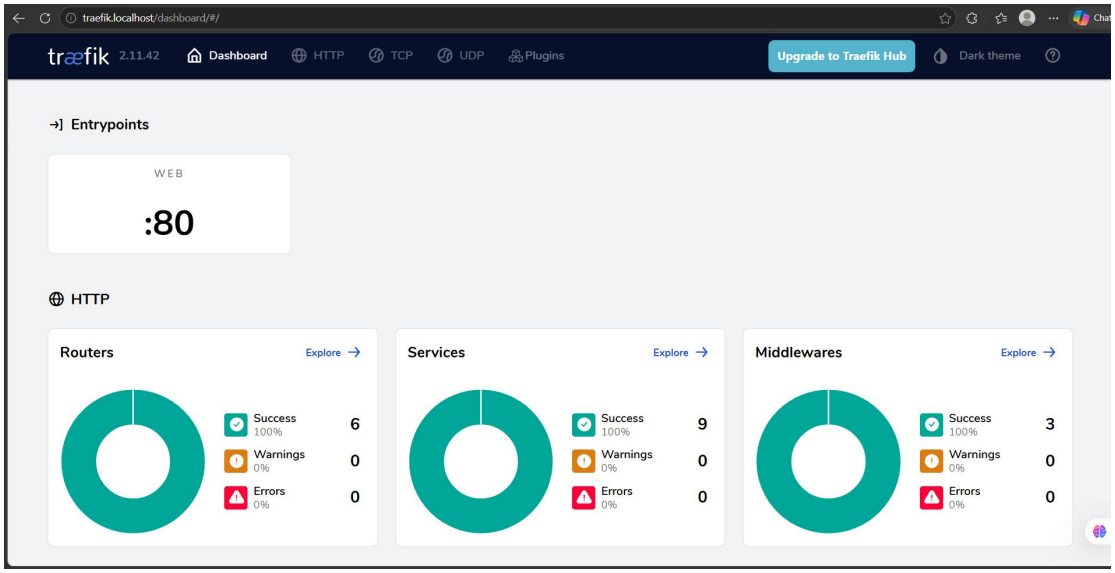
Yêu cầu:

1. Cấu hình Traefik với Docker provider.
2. Đặt labels cho services để Traefik nhận diện.

Tips khi làm bài tập:

1. Luôn dùng docker-compose down -v để xóa volumes khi test lại.
2. Kiểm tra log bằng docker-compose logs <service>.
3. Sử dụng docker-compose config để validate file YAML.




traefik.localhost/dashboard/#/
traefik 2.11.42
Dashboard
HTTP
TCP
UDP
Plugins
Upgrade to Traefik Hub
Dark theme
Entrypoints
WEB
:80
HTTP
Routers
Explore
Success 100% 6
Warnings 0% 0
Errors 0% 0
Services
Explore
Success 100% 9
Warnings 0% 0
Errors 0% 0
Middlewares
Explore
Success 100% 3
Warnings 0% 0
Errors 0% 0